

5. Comunicación entre procesos

Contenido

1. ¿Por qué no un solo proceso?	2
2. El modelo multi proceso	2
2.1. El proceso principal	2
2.1.1. Gestión de ventanas	2
2.1.2. Ciclo de vida de la aplicación	3
2.1.3. Aplicaciones nativas	4
2.2. El proceso de renderizado	4
3. Scripts de precarga	5
4. Comunicación entre procesos	6
4.1. Canales IPC	6
Patrón 1: Renderizador a principal (unidireccional)	7
Patrón 2: Principal a principal (bidireccional)	9
Patrón 3: Principal a renderizador	12
Patrón 4: Renderizador a renderizador	3
4.2. Serialización de objetos	3

Electron hereda su **arquitectura multiproceso de *Chromium***, lo que hace que el marco sea muy similar a un navegador web moderno.

1. ¿Por qué no un solo proceso?

Los navegadores web son aplicaciones increíblemente complicadas. Aparte de su capacidad principal para mostrar contenido web, tienen muchas responsabilidades secundarias, como administrar múltiples ventanas (o pestañas) y cargar extensiones de terceros.

Anteriormente, los navegadores solían utilizar un único proceso para toda esta funcionalidad. Aunque este patrón significaba menos sobrecarga para cada pestaña abierta, también significaba que un sitio web que fallaba o se bloqueaba afectaría a todo el navegador.

2. El modelo multi proceso

Para resolver este problema, el equipo de *Chrome* decidió que cada pestaña se mostraría en su propio proceso, limitando el daño que un código malicioso o con errores podría causar a la aplicación en su conjunto. Un único proceso del navegador controla estos procesos, así como el ciclo de vida de la aplicación en su conjunto. El diagrama de *Chrome Comic* visualiza este modelo.

Las aplicaciones *Electron* están estructuradas de manera muy similar. El desarrollador controla dos tipos de procesos: **principal** (*main process*) y **renderizado** (*renderer process*).

2.1. El proceso principal

Cada aplicación de *Electron* tiene **un único proceso principal**, que actúa como punto de entrada de la aplicación. El proceso principal **se ejecuta en un entorno *Node.js***, lo que significa que tiene la capacidad de requerir módulos y utilizar todas las API de *Node.js*.

2.1.1. Gestión de ventanas

El propósito principal del proceso principal es **crear y administrar ventanas** de aplicaciones con el módulo `BrowserWindow`.

Cada instancia de la clase `BrowserWindow` crea una ventana de aplicación que carga una página web en un **proceso de renderizado independiente**. Se puede interactuar con este contenido web desde el proceso principal utilizando el objeto `webContents` de la ventana.

main.js

```
const { app, BrowserWindow } = require('electron');

const createWindow = () => {
  const win = new BrowserWindow({ width: 800, height: 600 });
  win.loadURL('https://github.com');

  const contents = win.webContents;
  console.log(contents);
};

app.whenReady().then(() => {
  createWindow();
});
```

Nota:

También se crea un proceso de renderizado para webs incrustadas como el módulo `BrowserView`. El objeto `webContents` también es accesible para contenido web incrustado.

Debido a que el módulo `BrowserWindow` es un `EventEmitter`, también puede agregar controladores para varios eventos de usuario (*por ejemplo, minimizar o maximizar la ventana*). Cuando una instancia de `BrowserWindow` se destruye, su proceso de renderizado también finaliza.

2.1.2. Ciclo de vida de la aplicación

El proceso principal también controla el ciclo de vida de su aplicación a través del **módulo** de *Electron*. Este módulo proporciona un gran conjunto de eventos y métodos que se pueden utilizar para agregar un comportamiento personalizado de la aplicación (*por ejemplo, cerrar la aplicación mediante código, modificar el dock de la aplicación o mostrar un panel 'Acerca de'*).

Como ejemplo práctico, la aplicación que se muestra en la guía de inicio rápido utiliza **app** para crear una experiencia de ventana de aplicación más nativa.

main.js

```
// cerrando la aplicación cuando no queden ventanas abiertas en
// plataformas no macOS
app.on('window-all-closed', function () {
  if (process.platform !== 'darwin') app.quit();
});
```

2.1.3. Aplicaciones nativas

Para ampliar las funciones de *Electron* más allá de ser un contenedor de *Chromium* para contenidos web, el proceso principal también agrega API personalizadas para interactuar con el sistema operativo del usuario. *Electron* expone varios módulos que controlan la funcionalidad nativa del escritorio, como menús, cuadros de diálogo e íconos de bandeja del sistema.

Para obtener una lista completa de los principales módulos de proceso de *Electron*, se puede consultar la [documentación API](#).

2.2. El proceso de renderizado

Cada aplicación de *Electron* genera un proceso de renderizado independiente para cada `BrowserWindow` (y cada web embebida). Un renderizador es responsable de representar contenido web. A todos los efectos, el código ejecutado en los procesos de renderizado debe comportarse de acuerdo con los estándares web.

Por lo tanto, todas las interfaces de usuario y las funciones de las aplicaciones dentro de una única ventana del navegador deben escribirse con las mismas herramientas y paradigmas que utiliza en la web.

Lo mínimo que se debe comprender es:

- Un archivo HTML es su punto de entrada para el proceso de renderizado.
- El estilo de la interfaz de usuario se agrega mediante hojas de estilo (CSS o bootstrap).
- Se puede agregar código *JavaScript* ejecutable a través de etiquetas `<script>`.

Además, esto también significa que **el renderizador no tiene acceso directo a `require` o otras API de *Node.js***.



ATENCIÓN

Los procesos de renderizado se pueden generar con un entorno *Node.js* completo para facilitar el desarrollo. Históricamente, esta solía ser la opción predeterminada, pero esta función está deshabilitada por razones de seguridad.

Pero entonces, ¿cómo las interfaces de usuario del proceso de renderizado pueden interactuar con *Node.js* y la funcionalidad de escritorio nativa de *Electron* si solo se puede acceder a estas funciones desde el proceso principal? De hecho, no existe una forma directa de importar los *scripts* de contenido de *Electron*.

3. Scripts de precarga

Los *scripts* de precarga contienen código que se ejecuta en un proceso de renderizado antes de que su contenido web comience a cargarse. Estos scripts se ejecutan dentro del contexto del renderizador, pero se les otorgan más privilegios al tener acceso a las API de *Node.js*.

Se puede adjuntar un *script* de precarga al proceso principal en el constructor en

`BrowserWindow` la opción `webPreferences`.

`main.js`

```
const { app, BrowserWindow } = require('electron');
const path = require('node:path');

const createWindow = () => {
  const win = new BrowserWindow({
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
    },
  });
  win.loadFile('index.html');
};
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h1>hola</h1>
  </body>
  <script src="renderer.js"></script>
</html>
```

Debido a que el script de precarga comparte una interfaz `window` global con los renderizadores y puede acceder a las API de *Node.js*, sirve para mejorar su renderizador al exponer API arbitrarias en el `window` global que luego puede consumir su contenido web.

Aunque los scripts de precarga comparten un `window` global con el renderizador al que están adjuntos, no puede adjuntar directamente ninguna variable desde el script de precarga a `window` debido al valor predeterminado de `contextIsolation`.

preload.js

```
window.myAPI = { desktop: true };
```

renderer.js

```
console.log(window.myAPI);  
// => undefined
```

El aislamiento de contexto significa que los scripts de precarga están aislados del ámbito principal del renderizador para evitar la filtración de API privilegiadas en el código del contenido web.

En su lugar, se utiliza el módulo `contextBridge` para lograr esto de forma segura:

preload.js

```
const { contextBridge } = require('electron');  
contextBridge.exposeInMainWorld('myAPI', { desktop: true });
```

renderer.js

```
console.log(window.myAPI);  
// => desktop:true
```

Esta característica es increíblemente útil para dos propósitos principales:

- Al exponer las funciones de `ipcRenderer` al renderizador, puede utilizar la comunicación entre procesos (IPC) para activar las tareas principales del proceso desde el renderizador (y viceversa).
- Si se está desarrollando una aplicación de *Electron* para una aplicación web existente alojada en una URL remota, se pueden agregar propiedades personalizadas al `window` global del renderizador que se pueden usar para la lógica del escritorio en el lado del cliente.

4. Comunicación entre procesos

La comunicación entre procesos (IPC *Inter Process Communication*) es una parte clave de la creación de aplicaciones de escritorio. Debido a que los procesos principal y de renderizado tienen diferentes responsabilidades, IPC es la única forma de realizar muchas tareas, como llamar a una API nativa desde su interfaz de usuario o activar cambios en su contenido web desde menús nativos.

4.1. Canales IPC

Los procesos se comunican pasando mensajes a través de "canales" definidos por el desarrollador con los módulos `ipcMain` y `ipcRenderer`. Estos canales son **arbitrarios** (puedes nombrarlos como quieras) y **bidireccionales** (puedes usar el mismo nombre de canal para ambos módulos).

Repasaremos algunos patrones IPC fundamentales con ejemplos concretos que puedes usar como referencia para el código de tu aplicación.

Patrón 1: Renderizador a principal (unidireccional)

Para enviar un mensaje IPC unidireccional desde un proceso de renderizador al proceso principal, se puede usar `ipcRenderer.send` para enviar un mensaje que recibirá `ipcMain.on`.

Normalmente se utiliza este patrón para activar una acción en el proceso principal, como guardar un archivo o mostrar un cuadro de diálogo, desde una acción del usuario en el renderizador.

Para demostrar este patrón, crearemos una aplicación que utiliza IPC para actualizar el título de la ventana de la aplicación desde el proceso de renderizado.

1. Escuchar eventos con ipcMain.on

En el proceso principal, se crea un *listener* para el evento 'set-title' con `ipcMain.on`:

main.js

```
const { app, BrowserWindow, ipcMain } = require('electron');
const path = require('node:path');
function createWindow() {
  const mainWindow = new BrowserWindow({
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
    },
  });
  mainWindow.loadFile('index.html');
}
function handleSetTitle(event, title) {
  const webContents = event.sender;
  const win = BrowserWindow.fromWebContents(webContents);
  win.setTitle(title);
}
app.whenReady().then(() => {
  ipcMain.on('set-title', handleSetTitle);
  createWindow();
  app.on('activate', function () {
    if (BrowserWindow.getAllWindows().length === 0) createWindow();
  });
});
app.on('window-all-closed', function () {
  if (process.platform !== 'darwin') app.quit();
});
```

La función `handleSetTitle` tiene dos parámetros: un evento con estructura `IpcMainEvent` y una cadena de texto `title`. Cada vez que llega un evento `set-title`, esta función encontrará la instancia de `BrowserWindow` adjunta al remitente del mensaje y utilizará `win.setTitle` en ella.

2. Exponer `ipcRenderer.send` en el script `preload.js`

Para enviar mensajes al *listener* creado anteriormente, se puede usar `ipcRenderer.send`. Debemos elegir qué API exponer desde el script de precarga utilizando `contextBridge`.

El script de precarga expondrá el método `setTitle` en proceso de renderizado.

`preload.js`

```
const { contextBridge, ipcRenderer } = require('electron');
contextBridge.exposeInMainWorld('electronAPI', {
  setTitle: (title) => ipcRenderer.send('set-title', title),
});
```

En este punto, el proceso de renderizado puede llamar a `window.electronAPI.setTitle` para enviar un mensaje IPC al proceso principal.

3. Crear el proceso de renderizado en la interfaz de usuario

En el archivo HTML cargado en `BrowserWindows`, se añade una interface básica consistente en una entrada de texto y un botón.

`index.html`

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta
      http-equiv="Content-Security-Policy"
      content="default-src
'self'; script-src 'self'"
    />
    <title>Hello IPC!</title>
  </head>
  <body>
    Title: <input id="title" />
    <button id="btn" type="button">Set</button>
    <script src="./renderer.js"></script>
  </body>
</html>
```

Para hacer los elementos interactivos, añadimos algunas líneas de código a `renderer.js`, que aprovecha la API expuesta por el script de precarga.

renderer.js

```
const setButton = document.getElementById('btn');
const titleInput = document.getElementById('title');
setButton.addEventListener('click', () => {
  const title = titleInput.value;
  window.electronAPI.setTitle(title);
});
```

En este punto, se puede ejecutar la aplicación y probar el botón. Al hacer clic en el botón, se enviará un mensaje IPC al proceso principal, que establecerá el valor del campo de entrada en el título de la ventana.

Patrón 2: Principal a principal (bidireccional)

Para enviar un mensaje IPC bidireccional entre dos procesos, se puede usar `ipcMain.handle` para crear un *listener* que devuelva un valor. Luego, se puede usar `ipcRenderer.invoke` para enviar el valor devuelto.

Este patrón se puede utilizar para obtener información del proceso principal, como el nombre de usuario del sistema operativo, desde el proceso principal.

Para demostrar este patrón, crearemos una aplicación que utiliza IPC para abrir un cuadro de diálogo de sistema y devolver el archivo seleccionado al proceso de renderizado.

1. Crear un *listener* con `ipcMain.handle`

En el proceso principal, se crea la función `handleFileOpen()` que llama a `dialog.showOpenDialog` que mostrará un diálogo y devuelve el primer archivo seleccionado. Esta función se utiliza como *listener* cuando un mensaje `ipcRenderer.invoke` se envía con el evento `dialog:openFile` que escucha `ipcMain.handle`:

main.js

```
const { app, BrowserWindow, ipcMain, dialog } = require('electron');
const path = require('node:path');
async function handleFileOpen() {
  const { canceled, filePaths } = await dialog.showOpenDialog();
  if (!canceled) {
    return filePaths[0];
  }
}
```

```
function createWindow() {
  const mainWindow = new BrowserWindow({
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
    },
  });
  mainWindow.loadFile('index.html');
}
app.whenReady().then(() => {
  ipcMain.handle('dialog:openFile', handleFileOpen);
  createWindow();
  app.on('activate', function () {
    if (BrowserWindow.getAllWindows().length === 0) createWindow();
  });
});
app.on('window-all-closed', function () {
  if (process.platform !== 'darwin') app.quit();
});
});
```



dialog:

El prefijo `dialog:` no tiene efecto en el código. Se utiliza para mejorar la legibilidad.

2. Exponer `ipcRenderer.invoke` en el script `preload.js`

Para llamar a la función creada anteriormente, se puede usar `ipcRenderer.invoke`. Debemos elegir qué API exponer desde el script de precarga utilizando `contextBridge`.

El script de precarga expondrá el método `openFile` en proceso de renderizado.

preload.js

```
const { contextBridge, ipcRenderer } = require('electron');
contextBridge.exposeInMainWorld('electronAPI', {
  openFile: () => ipcRenderer.invoke('dialog:openFile'),
});
```

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta
      http-equiv="Content-Security-Policy"
      content="default-src
'self'; script-src 'self'"
    />
    <title>Dialog</title>
  </head>
  <body>
    <button type="button" id="btn">Open a File</button>
    File path: <span id="filePath"></span>
    <script src="./renderer.js"></script>
  </body>
</html>
```

3. Crear el proceso de renderizado en la interfaz de usuario

Finalmente, creamos el HTML a cargar en `BrowserWindows`, que contiene un botón que, al hacer clic, llamará a `window.electronAPI.openFile` y mostrará el resultado en un elemento `<span>`.

La interfaz consiste en un botón que utilizaremos para enviar un mensaje IPC al proceso principal y un elemento `<span>` que utilizaremos para mostrar el resultado. Para hacer los elementos interactivos, añadimos algunas líneas de código a `renderer.js`, que aprovecha la API expuesta por el script de precarga.

renderer.js

```
const btn = document.getElementById('btn');
const filePathElement = document.getElementById('filePath');
btn.addEventListener('click', async () => {
  const filePath = await window.electronAPI.openFile();
  filePathElement.innerText = filePath;
});
```

En el código de arriba, se utiliza `await` para esperar el resultado de `window.electronAPI.openFile()` y luego se establece el resultado en el elemento `<span>`.

Patrón 3: Principal a renderizador

Para enviar un mensaje desde el proceso principal a un proceso de renderizado, necesitamos especificar qué ventana de renderizado recibirá el mensaje. Los mensajes se envían a través de `WebContents` que se pueden obtener de `BrowserWindow`. Esta instancia de `WebContents` contiene un método `send` que puede usarse del mismo modo que `ipcRenderer.send` en el proceso de renderizado.

Para demostrar este patrón, crearemos un contador de números controlado por el menú nativo del sistema operativo.

1. Enviar mensajes con el módulo `webContents`

Para esta demo, necesitamos crear un menú personalizado en usando el módulo `Menu` que usa `webContents.send` para enviar un mensaje IPC desde el proceso principal al proceso renderizador objetivo.

`main.js`

```
const { app, BrowserWindow, Menu, ipcMain } = require('electron');
const path = require('node:path');
function createWindow() {
  const mainWindow = new BrowserWindow({
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
    },
  });
}
const menu = Menu.buildFromTemplate([
  {
    label: app.name,
    submenu: [
      {
        click: () => mainWindow.webContents.send('update-counter', 1),
        label: 'Increment',
      },
      {
        click: () => mainWindow.webContents.send('update-counter', -1),
        label: 'Decrement',
      },
    ],
  },
]);
Menu.setApplicationMenu(menu);
mainWindow.loadFile('index.html');
// Open the DevTools.
mainWindow.webContents.openDevTools();
}
```

```

app.whenReady().then(() => {
  ipcMain.on('counter-value', (_event, value) => {
    console.log(value); // will print value to Node console
  });
  createWindow();
  app.on('activate', function () {
    if (BrowserWindow.getAllWindows().length === 0) createWindow();
  });
});
app.on('window-all-closed', function () {
  if (process.platform !== 'darwin') app.quit();
});

```

2. Exponer ipcRenderer.on en el script preload.js

Como en los ejemplos anteriores, usamos los módulos contextBridge y ipcRenderer para exponer la funcionalidad IPC en el proceso de renderizado.

preload.js

```

const { contextBridge, ipcRenderer } = require('electron');
contextBridge.exposeInMainWorld('electronAPI', {
  onUpdateCounter: (callback) => ipcRenderer.on('update-counter', callback),
});

```

3. Crear el proceso de renderizado en la interfaz de usuario

Para terminar de unirlo todo, creamos la interfaz HTML que contiene un elemento `<span>` que se actualizará con el valor del contador.

index.html

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta
      http-equiv="Content-Security-Policy"
      content="default-src
'self'; script-src 'self'"
    />
    <title>Menu Counter</title>
  </head>
  <body>
    Current value: <span id="counter">0</span>
    <script src="./renderer.js"></script>
  </body>
</html>

```

Finalmente, para hacer que los valores se actualicen en el documento HTML, añadimos algunas líneas de código a `renderer.js`, que aprovecha la API expuesta por el script de precarga.

```
renderer.js
const counter = document.getElementById('counter');
window.electronAPI.onUpdateCounter((_event, value) => {
  const oldValue = Number(counter.innerText);
  const newValue = oldValue + value;
  counter.innerText = newValue.toString();
});
```

En el código anterior, se utiliza `window.electronAPI.onUpdateCounter` para registrar un *listener* para el evento `update-counter` que actualiza el valor del elemento `<span>`.

Patrón 4: Renderizador a renderizador

No existe una forma directa para enviar mensajes entre procesos de renderizado. Para conseguirlo tenemos dos opciones:

- Usar el proceso principal como intermediario. Esto implica enviar un mensaje desde cada renderer al proceso principal, que reenviará el mensaje al renderer objetivo.
- Pasar un `MessagePort` desde el proceso principal a los dos renderers. Esto permite una comunicación directa entre los procesos renderizadores después de una configuración inicial.

4.2. Serialización de objetos

La implementación IPC utiliza el estándar HTML `Structured Clone Algorithm` para serializar los objetos pasados entre procesos. Esto significa que los objetos que se envían a través de IPC deben ser compatibles con este algoritmo.

Puedes enviar sin problemas objetos simples (`{}`, `[]`, `String`, `Number`, `Boolean`), fechas (`Date`), mapas (`Map`), conjuntos (`Set`), `ArrayBuffer` y clases personalizadas simples (sin métodos especiales).

No puedes enviar objetos DOM (ej. `Element`, `Location` y `DOMMatrix`), objetos Node.js con clases C++ (ej. `process.env`, miembros de `Stream`), y objetos Electron con clases C++ (ej. `WebContents`, `BrowserWindow` y `WebFrame`) ya que no son serializables con el algoritmo de clonación estructurada.